Time to Detected Contradiction in Saved AI Code Explanations

Anonymous Author(s)

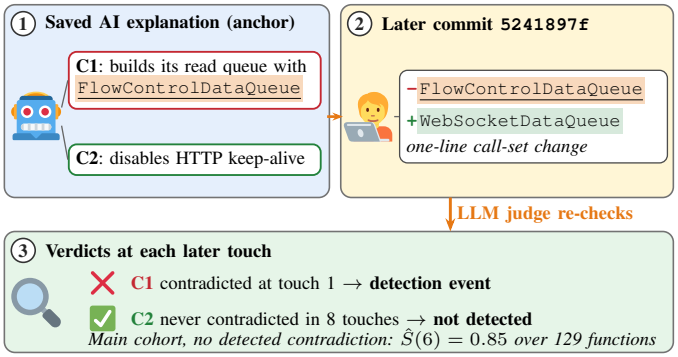


Fig. 1. Hand-picked aiohttp case, `_post_start`, touch 1 = `5241897f` (shortened claims). The judge contradicts C1; C2 remains uncontradicted.

***Abstract*—AI code explanations are increasingly saved in repositories, yet their accuracy is usually assessed only at creation. We follow individual claims as their functions change, recording the first contradiction detected by a specified judge. In 129 functions from 11 Python projects, Qwen leaves an estimated 0.85 of initially supported claims without a detected contradiction after six code-changing commits (95% CI 0.81–0.87); 10% receive a detection by the fourth commit. Validation substantially changes this picture: a full Llama replay gives $\hat{S}(6) = 0.613$, while substituting one independent rater's labels for observed Qwen events gives 0.923–0.925. These analyses use different judge-conditioned risk sets or event rules and do not bound semantic accuracy. Larger edits are associated with more detections, but provide only moderate discrimination. The contribution is a reproducible way to study when retained claims trigger review, together with evidence that judge calibration is essential before detection curves can guide documentation maintenance.**

***Index Terms*—code explanation, program comprehension, documentation decay, survival analysis, LLM**

I. Introduction

LLM-generated explanations of code are increasingly kept rather than discarded: tools such as RepoAgent [1], DocAgent [2] and RepoDoc [3] write them into repository documentation, and developers paste them into reviews and wikis. The code keeps changing; the saved text does not. Whether such text is still true later is a program-comprehension question, because a fluent but stale statement seeds a wrong mental model [4]. Yet explanations are evaluated almost only when they are produced [5]–[7], and human-comment research treats a comment as one unit that is consistent or not [8], [9].

An explanation need not become outdated all at once: a changed exception can invalidate one sentence while leaving the description of the return value intact. We therefore follow individual *claims* through later versions of their function (Fig. 1). A *touch* is a commit that changes the function's code. We record each claim's first judge-detected contradiction and estimate $S(k)$, the proportion remaining free from detection after k touches, allowing for incomplete follow-up. This makes the unit of maintenance explicit: which retained claim first warrants review, and after how much code change?

We study three questions: when does the primary judge first flag retained claims; how much does the answer change under alternative judgments; and whether edit size or claim type offers useful review signals. The strongest finding is the dependence of the curve on how contradictions are measured. Our results motivate calibrating the detector before using such curves to set refresh schedules. Throughout, the endpoint is detected contradiction; semantic failure time and developer benefit remain unmeasured.

The study began with qualitative expectations about churn, claim type, whole-text checks and docstrings, but both the operational tests and the implementation plan changed after data collection began (Section IV). We therefore report every covariate and comparator result as exploratory diagnostics, rather than as confirmatory hypothesis tests.

Our distinction from the closest work is temporal: Khatib et al. [5] test newly generated summaries after synthetic mutations; we retain the original explanation and follow its claims over real commits. Earlier documentation-decay and refresh studies [10]–[12] motivate this setting.

Contributions. (1) An operational detection-time measure with censoring and explicit observation assumptions. (2) Cached longitudinal records, model and single-rater human-label sensitivity analyses, and a two-human-rater audit with blind adjudication. (3) Exploratory diagnostics and a validation agenda, including frozen-input detector comparisons and adjudication-aware survival estimation.

II. Background and Related Work

Decay of human documentation. Comments and code often fail to co-evolve [13]; stale comments mislead developers [14]; Wen et al. catalogued the code-change types that introduce comment inconsistencies [8]. Radmanesh et al. follow inconsistencies over time and find their bug risk highest right after they appear [15]. Comment links decay [10], outdated code-element references are widespread [11], and documentation churn trails code churn [16]. Survival analysis is established in software engineering, e.g. for self-admitted technical debt [17] and for AI-generated code [18]. We study

the decay of *retained AI explanations* at the level of individual claims, which these works do not.

Just-in-time (JIT) detection. Detectors decide per commit whether a comment became inconsistent [9], [19]. CARL-CCI uses structured diffs [20]; CCISolver uses LLMs [21]. Execution-based approaches provide a different triangulation: CASCADE requires a documentation-derived test to fail on the implementation and pass on synthesized code [22]. Our exploratory whole-text prompt diagnostic is not a test of these published systems; a frozen-input comparison on the same eligible function–touch inputs remains future work.

Faithfulness of generated summaries. Khatib et al. inject behaviour-changing mutations and test whether *newly generated* LLM summaries reflect them [5]. ETF [7] and ReFEree [6] decompose summaries into checkable units, as does atomic-fact scoring [23]; Sistla et al. check formalised code-reasoning claims with program analysis [24]. We apply claim decomposition *longitudinally* to an explanation that is kept, over its function's history.

Uncertain event labels. Conventional Kaplan–Meier treats recorded events as correct. Recent work on contaminated and adjudicated events instead incorporates an explicit expert-error mechanism into conditional survival estimation [25]. Our human-label substitution is deliberately weaker: it removes observed primary-judge false positives, but supplies neither a validated contamination model nor information on missed events; it is thus a sensitivity analysis, not that estimator.

Keeping documentation current. RepoAgent [1], DocAgent [2], RepoDoc [3] and CODENS [12] update repository documentation; CoDocBench pairs code/docstring changes [26]. READU checks README inconsistencies against internal/external facts [27]. Such grounding and ReFEree's dependency-aware segments [6] could strengthen our local judge; neither is evaluated here. Our target is detection time for retained claims.

III. Approach: Claim-Survival Analysis

Construct and decomposition. An explanation E of function f is written at anchor c_0. The prompt requires each claim to be a self-contained, source-checkable fact naming f, with no added or merged facts, original wording where possible, and a 12-claim cap. Each receives one type: *structural* (signature, parameters, calls), *data-flow*, *behavioural* (conditions, exceptions, side effects), or *intent*. These instructions do not guarantee atomicity or coverage. A *touch* v_k is a later mainline commit changing f's code tokens (docstrings, comments and layout removed). Git file renames are followed; fallback matching requires a unique new same-name candidate in the same file with text similarity ≥ 0.7. This is an operational tracking rule, not validated identity resolution: false matches may create spurious follow-up, while failed matches censor tracks and can be informative. A judge $J(c, f@v_k) \in \{\text{SUP}, \text{CON}, \text{UNR}\}$ uses docstring-free code, the diff and tree-sitter [28] facts (parameters, calls, returns, raises).

Cohort, event, censoring. Only anchor-SUP claims enter; anchor contradictions are initial detected errors. The event is the *first* CON, so later support cannot undo it. Follow-up is censored at $K_{\max}$, the pinned head, or unmatched removal/rename. Kaplan–Meier [29] estimates $S(k) = \Pr[T > k]$ for this detection time, conditional on anchor support. We report $\hat{S}(K)$ and the time to 10% detections; the half-life is not reached. *Unresolved verdicts:* primary analysis keeps UNR claims at risk without an event; sensitivity scenarios instead censor immediately before the first UNR, or count it as an event. Neither scenario bounds semantic survival; unresolved is not supported. Formal error adjustment needs judge sensitivity/specificity that this archive cannot identify. Follow-up stops at the first detection, precluding analysis of later recovery.

Explaining detections. To explore which edits accompany detections, we fit a discrete-time complementary log-log model with a separate baseline for each touch index. Predictors are claim type, $\log(1+\text{lines changed})$ in the function and file, and change flags derived from parser facts. A hazard ratio (HR) describes the relative detection rate among claims still at risk. Intervals and Wald tests account for claims sharing a function. Cox models with changing covariates [30]–[32] and proportional-hazards checks are retained as robustness analyses in the artifact.

Placebo sensitivity. Rejudging unchanged code gives a scenario with constant false-contradiction rate ϵ: $\max(0, (h_k - \epsilon)/(1 - \epsilon))$. Error adjustment would require placebo rates to transfer to real-diff prompts, independent repeated errors conditional on risk, and no missed contradictions. Contexts differ and judges disagree both ways; these assumptions are unvalidated. The curve is a sensitivity scenario, not a calibrated correction.

Exploratory comparators. A whole-explanation JIT check by the same model uses three prompt versions (binary v1 and v2, graded 0–10 v3); a non-LLM *identifier check* flags a claim once an identifier it names, present at c_0, is missing from the code. We also decompose and judge the function's anchor docstring in the same shuffled list as the LLM claims. These are implementation diagnostics, not published-baseline comparisons or causal authorship tests.

IV. Preliminary Evaluation

Subjects and units. We cloned 14 Python projects, pinned their heads, and anchored explanations at the last mainline commit before 2024-11-01. The main cohort comprises 129 non-test functions from 11 projects (6–120 lines, at least 6 future touches, seeded project cap of 20). Follow-up extends to 12 touches (Table I). The count ledger distinguishes 1038 candidate function–commit touches, 1020 effective touches after formatting exclusions, and 1014 at-risk function–touch rows used for churn analysis. Duplicate names and overload stubs are excluded; one later stub ends follow-up. To examine selection on future activity, we also sampled 80 eligible anchor functions, leaving 61 after vendored-code exclusions; 21 are ever touched. Full eligibility, exclusions and a supplemental clean redraw are archived.

TABLE I

SUBJECTS. MAIN COHORT: FUNCTIONS, FUNCTION–COMMIT TOUCHES (UP TO 12 EACH) AND DISTINCT COMMITS; RAND.: RANDOM-COHORT FUNCTIONS (OTHERS: RICH, FLASK, HTTPX).

Project	Func.	Touches	Commits	w/ doc	LLM cl.	Doc cl.	Rand.
pytest	9	66	51	4	94	30	5
pydantic	20	171	96	18	223	125	2
black	15	123	88	10	165	36	2
click	5	41	28	4	56	18	3
attrs	1	6	6	1	9	5	0
pip	16	125	69	4	181	16	7
hypothesis	19	172	95	5	196	25	6
sphinx	12	96	64	3	131	7	11
pylint	3	24	23	3	32	4	8
aiohttp	20	150	103	9	227	21	0
textual	9	64	48	4	101	23	12
others	–	–	–	–	–	–	5
Total	129	1038	671	65	1415	310	61

TABLE II

FREEDOM FROM JUDGE-DETECTED CONTRADICTION AMONG ANCHOR-SUPPORTED CLAIMS. MAIN COHORT SELECTS FUTURE-ACTIVE, TRACKABLE FUNCTIONS; PLACEBO ADJUSTMENT IS A SCENARIO. HRS VS. DATA-FLOW.

Claims	n	Events$_{\le 12}$	$\hat{S}(6)$ [95% CI]	$t_{0.9}$	HR [95% CI]
Main (all)	1354	250	0.85 [0.81, 0.87]	4	–
placebo scenario			0.86 [0.83, 0.89]	–	
Random (k=1)	219	12	n.r.	–	–
Structural	443	93	0.81 [0.77, 0.85]	3	1.06 [0.81, 1.38]
Data-flow	489	102	0.84 [0.80, 0.87]	5	1 (ref.)
Behavioural	338	49	0.88 [0.84, 0.91]	5	0.73 [0.51, 1.05]
Intent	84	6	0.93 [0.87, 0.99]	n.r.	0.32 [0.13, 0.79]

Models. One model, qwen2.5-coder:14b [33] (temperature 0, cached), generates explanations (from code *with* docstring, prompted to cover parameters, helpers and errors), decomposes, judges and runs the JIT checks; 72 of 129 explanations hit the 12-claim cap, and the judge prompt ("written about an earlier version") may prime. As a *reliability proxy, not human annotation*, claude-haiku-5-5 re-judged identical claim lists (96 of 125 pairs still valid, a stability-selected subset).

Protocol changes. The anchor date, projects, touch threshold, project cap, judging format/context, JIT prompts, analysis models, random cohort, placebos, and exclusions changed after the study began. The complete dated change log and frozen inputs are in the artifact. Consequently, we do not treat p-values, Holm adjustment, or any stated decision rule as confirmatory evidence. The archived ledger contains 3818 local and 205 hosted calls; the separate reviewer-follow-up replay and mutation calls are reported in the artifact experiment ledgers.

Initial judge-detected errors. At the anchor the judge contradicted 1.3% of LLM claims and 13.9% of docstring claims. An older assistant-only audit lacks its claim-level labels and is not used for any result here. The separately released human-audit records, matrix and sampling weights are available in the artifact.

When detections appear. Table II and Fig. 2 describe the future-active, trackable main cohort. Of 1354 anchor-supported claims, 250 receive a contradiction over follow-up to touch 12. At six touches, $\hat{S}(6) = 0.85$ [0.81,0.87]; the 10% detection threshold is reached at touch 4. By touch 12, $\hat{S} = 0.76$, so the median detection time is not observed. These results show detections emerging after initially supported explanations have been retained; they do not establish when the underlying claims became false. By touch six, 208 detections occur in 150 function–touch groups, including 97 in 39 groups with multiple detected claims, reinforcing the need to account for shared functions.

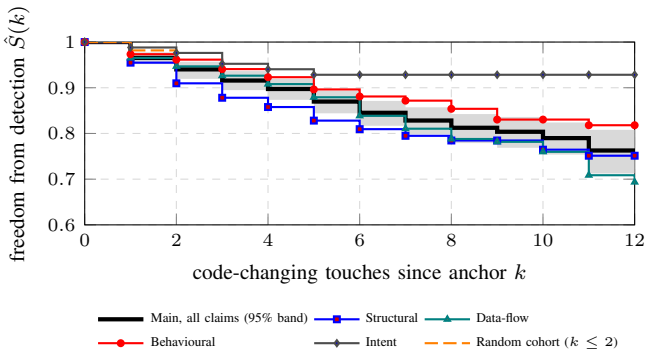


Fig. 2. Kaplan–Meier freedom from judge-detected contradiction: future-active main cohort (all, cluster-bootstrap band, by type) and random cohort.

Activity and observation matter. The random cohort contributes only 219 claims in 21 touched functions. By touch three, only 24 claims in 2 functions remain at risk; under our minimum of 30 claims in five functions, we report its curve only through touch 2 ($\hat{S} = 0.97$). Its high one-year freedom from detection (0.98, versus 0.89 in the main cohort) therefore cannot be read as stronger claim durability: many functions receive no local edit. Counting removal/rename as an event changes the random estimate to 0.92 [0.84,0.98]. With few affected functions and selection on future activity, these contrasts do not establish a population ordering. Full calendar-time and competing-risk analyses are archived.

Sensitivity to observation rules. Censoring before the first unresolved verdict gives $\hat{S}(6) = 0.849$; counting unresolved as an event gives 0.817 (primary 0.85; 53 affected claims). The choice matters, although these estimates remain much closer to each other than to the Llama replay below. Unchanged-code checks yield 5 contradictions in 1780 looks, all in anchor retests; real unchanged versions yield 0/426. The constant-error placebo scenario gives $\hat{S}(6) = 0.86$ [0.83,0.89]. This small change is conditional on error transfer from unchanged to changed code; the human event review demonstrates why that assumption cannot calibrate the primary curve.

Which edits warrant attention? Larger function edits are associated with more Qwen detections: HR 1.56 [1.19,2.03] per inter-quartile range of lines changed. However, AUC 0.68 indicates only moderate discrimination over 1014 analysis rows, 82.5% of which have no event. Churn may help prioritize review, but these results do not establish an effective automatic refresh trigger. Claim-type contrasts are less stable: the appar-

TABLE III
TWO-RATER AUDIT REFERENCE OF 400 RECORDED JUDGMENTS. ROWS ARE BULK-MODEL LABELS; COLUMNS ARE FINAL AUDIT LABELS.

	SUP	CON	UNR
Bulk SUP	132	1	0
Bulk CON	0	134	0
Bulk UNR	28	2	103

ent intent advantage (HR 0.32 [0.13,0.79] against data-flow) rests on 6 events, 4 in three click pager functions. We therefore infer no ordering by claim type.

Whole-text prompts v1/v2 flag every queried touch, including unchanged anchors; v3 barely discriminates (AUC 0.58 [0.55,0.61]). This failed prompt diagnostic leaves comparison with published JIT detectors open. The docstring contrast is also descriptive because generator exposure, claim granularity and initial eligibility differ. All coefficients and negative results remain in the artifact.

Reliability and human audit. Reweighted to the population strata (155/619 touches with/without a death), the archived judges agree on 65.9% of verdicts ($\kappa = 0.09$; $\kappa = 0.21$ for contradicted vs. not). claude-haiku-5-5 confirms 51.9% of bulk contradictions and contradicts 6.7% of claims the bulk judge kept: this proxy disagrees in both directions and cannot calibrate $\hat{S}$. Separately, two self-reported blinded raters audited the same 400 records stratified by stage and bulk verdict. They agree on 99.50% ($\kappa = 0.992$; weighted agreement 99.980%), with 2 disagreements resolved by a self-reported independent blinded adjudicator (Table III). The resulting reference labels all 134 sampled bulk-CON records as CON, yielding observed agreement 134/134 (exact 95% lower limit 97.3%) in that sampled stratum; among 297 resolved reference labels, bulk-CON sensitivity is 134/137=97.8%. These are not semantic PPV/recall: post-plan sampling, unresolved labels, self-reported blinding, and possible rater bias prevent that interpretation.

V. DISCUSSION AND RESEARCH AGENDA

Measurement changes the maintenance story. Under Qwen, 10% of initially supported claims receive a detection by touch four, and roughly 15% by touch six. A full Llama replay of the same 129 functions gives $\hat{S}(6) = 0.613$ [0.573,0.651] among 1290 Llama-supported anchor claims, compared with 1354 under Qwen. Anchor agreement is 87.19% ($\kappa = 0.241$; 1,725 labels). The replay changes both eligibility and subsequent judgments, so it cannot isolate follow-up error alone. The gap would nevertheless materially affect a refresh schedule; choosing one from the primary curve would outrun the validation evidence.

Human review changes the event count. An independent rater relabeled all 218 unique observed Qwen-CON rows through touch six: 102 contradicted, 114 supported and 2 unresolved. Confirmation among resolved rows is 47.2% [40.7,53.9% Wilson]. Substitution yields $\hat{S}(6) = 0.923$ when unresolved is an event or 0.925 when censored (function-cluster bootstrap 0.904–0.940). It removes observed false positives but cannot recover missed contradictions, including after a rejected first event in the alive-only archive. The increase demonstrates the consequence of event verification; it does not establish a corrected semantic curve.

The two-rater audit and event review answer different questions. A1/A2 agree on 99.50% of 400 records ($\kappa = 0.992$), yet the event rater confirms only 23 of 55 shared Qwen-CON rows. Agreement within one frame therefore does not establish calibration across raters and frames. Independence and blinding are self-reported; no calibration session is documented. A next audit should use common guidelines and independently adjudicate both flagged and unflagged claims.

Controlled checks have limited reach. A synthetic suite detects 100.0% of 100 known changes [96.3–100% Wilson] with 0.0% false positives among 100 unchanged controls. Yet targeted review supports 75/77 Qwen-SUP/Llama-CON anchor rows and contradicts only 9/37 Qwen-CON/Llama-SUP rows. Success on controlled mutations can coexist with ambiguity in repository histories. These selected checks do not estimate population precision or recall. Likewise, churn offers a candidate review priority, but moderate discrimination and dependence on Qwen labels leave operational value open.

What to validate next. First, compare published JIT detectors and execution-based checks on frozen eligible inputs, reporting coverage as well as errors. Second, independently validate tracking across refactorings and renames: false matches create spurious follow-up, while failed matches may censor informatively. Third, collect representative adjudication of flagged and unflagged claims before fitting contamination-aware survival methods [25]; the present data do not identify that error mechanism. Future-conditioned selection, sparse random-cohort events, 11 Python projects, one generator/prompt and non-local dependency changes constrain generalization. A larger, prospectively fixed cohort is needed before testing refresh policies or developer outcomes.

VI. CONCLUSION

Following retained claims makes their review timing measurable. Detections accumulate over successive edits, but model replay and human event review substantially change the estimated curve. Detector validation must precede maintenance policy. The released histories and analyses provide a starting point for that validation and for studies of targeted explanation refresh.

Data availability. Anonymous code and data: https://claim-survival-artifact.dimpledshrimp2.chatgpt.site.

AI-USE DISCLOSURE

LLMs (qwen2.5-coder:14b, Llama 3.1 8B via Ollama and claude-haiku-5-5) are experimental subjects and judges. An AI coding assistant implemented the artifact, audits and draft; statistics are rendered from saved outputs, and AI audits are not human evaluation.

References

[1] Q. Luo, Y. Ye, S. Liang, Z. Zhang, Y. Qin, Y. Lu, Y. Wu, X. Cong, Y. Lin, Y. Zhang, X. Che, Z. Liu, and M. Sun, "RepoAgent: An LLM-powered open-source framework for repository-level code documentation generation," in *Proc. EMNLP: System Demonstrations*, 2024.

[2] D. Yang, A. Simoulin, X. Qian, X. Liu, Y. Cao, Z. Teng, and G. Yang, "DocAgent: A multi-agent system for automated code documentation generation," in *Proc. ACL: System Demonstrations*, 2025.

[3] D. Xu, M. Liu, X. Wang, J. Zhong, and Z. Zheng, "RepoDoc: A knowledge graph-based framework to automatic documentation generation and incremental updates," arXiv:2604.26523, 2026.

[4] A. von Mayrhauser and A. M. Vans, "Program comprehension during software maintenance and evolution," *IEEE Computer*, vol. 28, no. 8, pp. 44–55, 1995.

[5] L. Khatib, M. Pu, B. Vasilescu, and M. Nagappan, "Using mutation-analysis to examine an LLM's ability to summarize code," arXiv:2602.17838, 2026.

[6] S. Bae, C. Na, J. Lee, Y. Lee, Y. Choi, and J.-H. Lee, "ReFEree: Reference-free and fine-grained method for evaluating factual consistency in real-world code summarization," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026, arXiv:2604.10520.

[7] K. Maharaj, V. Munigala, S. G. Tamilselvam, P. Kumar, S. Sen, P. Kodeswaran, A. Mishra, and P. Bhattacharyya, "ETF: An entity tracing framework for hallucination detection in code summaries," arXiv:2410.14748, 2024.

[8] F. Wen, C. Nagy, G. Bavota, and M. Lanza, "A large-scale empirical study on code-comment inconsistencies," in *Proc. IEEE/ACM Int. Conf. Program Comprehension (ICPC)*, 2019, pp. 53–64.

[9] S. Panthaplackel, J. J. Li, M. Gligoric, and R. J. Mooney, "Deep just-in-time inconsistency detection between comments and source code," in *Proc. AAAI Conf. Artificial Intelligence*, 2021.

[10] H. Hata, C. Treude, R. G. Kula, and T. Ishio, "9.6 million links in source code comments: Purpose, evolution, and decay," in *Proc. IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 1211–1221.

[11] W. S. Tan, M. Wagner, and C. Treude, "Detecting outdated code element references in software repository documentation," *Empirical Software Engineering*, vol. 29, no. 1, p. 5, 2024.

[12] A. Kelious, C. Tahri, and E. Bardet, "CODENS: Transforming code changes into living, accessible, and queryable documentation," arXiv:2607.18356, 2026.

[13] B. Fluri, M. Würsch, and H. C. Gall, "Do code and comments co-evolve? On the relation between source code and comment changes," in *Proc. Working Conf. Reverse Engineering (WCRE)*, 2007, pp. 70–79.

[14] L. Tan, D. Yuan, G. Krishna, and Y. Zhou, "/*iComment: Bugs or bad comments?*/," in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2007, pp. 145–158.

[15] S. Radmanesh, A. Imani, I. Ahmed, and M. Moshirpour, "Investigating the impact of code comment inconsistency on bug introducing," arXiv:2409.10781, 2024.

[16] A. Hovhannisyan, Y. Fan, G. Rodriguez-Perez, and R. G. Kula, "Linking code and documentation churn: Preliminary analysis," arXiv:2410.05992, 2024.

[17] B. A. Muse, C. Nagy, A. Cleve, F. Khomh, and G. Antoniol, "FIXME: Synchronize with database! An empirical study of data access self-admitted technical debt," *Empirical Software Engineering*, vol. 27, no. 6, p. 130, 2022.

[18] M. Rahman and E. Shihab, "Will it survive? Deciphering the fate of AI-generated code in open source," arXiv preprint arXiv:2601.16809, 2026.

[19] Z. Liu, X. Xia, D. Lo, M. Yan, and S. Li, "Just-in-time obsolete comment detection and update," *IEEE Trans. Software Engineering*, vol. 49, no. 1, pp. 1–23, 2023.

[20] P. Nguyen, A. M. T. Bui, and P. T. Nguyen, "Larger is not always better: Leveraging structured code diffs for comment inconsistency detection," arXiv:2512.19883; SANER 2026 short paper, 2026.

[21] R. Zhong, Y. Huo, W. Gu, J. Kuang, Z. Jiang, G. Yu, Y. Li, D. Lo, and M. R. Lyu, "CCISolver: End-to-end detection and repair of method-level code-comment inconsistency," arXiv:2506.20558, 2025.

[22] T. Kiecker, J. A. Sparka, M. Reuter, A. Ziegler, and L. Grunske, "CASCADE: Detecting inconsistencies between code and documentation with automatic test generation," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, p. FSE168, 2026.

[23] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[24] M. Sistla, G. Balakrishnan, P. Rondon, J. Cambronero, M. Tufano, and S. Chandra, "Towards verified code reasoning by LLMs," arXiv:2509.26546, 2025.

[25] M. Bladt and K. V. Dinesen, "Nonparametric survival estimation with contaminated and adjudicated events," *Scandinavian Journal of Statistics*, vol. 53, no. 4, pp. 1520–1537, 2026.

[26] K. Pai, P. Devanbu, and T. Ahmed, "CoDocBench: A dataset for code-documentation alignment in software maintenance," in *Proc. Int. Conf. Mining Software Repositories (MSR), Data and Tool Showcase Track*, 2025.

[27] D. Baek, K. Krampf, and M. Pradel, "READU: Inconsistency-driven just-in-time detection and repair of README bugs," arXiv:2607.15780, 2026.

[28] "Tree-sitter documentation," https://tree-sitter.github.io/tree-sitter/, accessed: November 11, 2026.

[29] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *J. American Statistical Association*, vol. 53, no. 282, pp. 457–481, 1958.

[30] D. R. Cox, "Regression models and life-tables," *J. Royal Statistical Society: Series B*, vol. 34, no. 2, pp. 187–202, 1972.

[31] P. K. Andersen and R. D. Gill, "Cox's regression model for counting processes: A large sample study," *The Annals of Statistics*, vol. 10, no. 4, pp. 1100–1120, 1982.

[32] C. Davidson-Pilon, "lifelines: Survival analysis in Python," *J. Open Source Software*, vol. 4, no. 40, p. 1317, 2019.

[33] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu *et al.*, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.